

FINAL PROJECT REPORT

Design and Implementation of an Agentic AI-Based Personal Task Automation System

A lightweight Discord-controlled automation agent inspired by OpenClaw

Planning date	February 19, 2026
Implementation period	June 20–21, 2026
Development team	One developer
Target platform	MacBook Air M1, macOS
Project status	MVP implemented and operational

Prepared for final project submission

June 21, 2026

Executive Summary

This project designed and implemented a lightweight personal AI agent that converts natural-language Discord messages into controlled automation actions. The system was built from scratch in Python rather than installing OpenClaw. It runs continuously on a MacBook Air M1 and integrates a hosted GPT model with a restricted set of tools for PDF OCR, Gmail management, Google Sheets task reading, Google Calendar inspection, and daily productivity briefings.

Final result

The MVP is operational. The complete automated test suite reports 21 passed tests, real OCR files were generated successfully, Gmail organization processed a specified date range, and the Discord bot remains online through a macOS launchd service.

Component	Status	Evidence
Discord interface	Complete	Owner-only natural-language commands; status messages and chunked replies
OCR automation	Complete	Korean/English text, LaTeX math, Markdown tables, figure descriptions
Gmail management	Complete	Explicit date range, classification, safe Trash action, summary report
Reply workflow	Complete	Draft ID with explicit send/cancel confirmation
Daily briefing	Complete	Gmail, Sheets, and Calendar summarized directly in Discord
24/7 operation	Complete	macOS launchd RunAtLoad + KeepAlive; AC sleep disabled
Automated tests	21 passed	Security, routing, ranges, reports, OCR and controller behavior

1. Project Background and Motivation

The project was planned on February 19, 2026 after the developer encountered early-2026 demonstrations of OpenClaw and overseas creators using always-on AI agents to automate routine digital work. These demonstrations suggested a transition from passive question-answering systems to agents that can interpret intent, call tools, and report completed actions through ordinary messaging applications.

The purpose of this project was not to reproduce or install OpenClaw. Instead, it used the concept as architectural inspiration and implemented a deliberately smaller system whose components, safety boundaries, and behavior could be understood and demonstrated by a single developer on consumer hardware.

1.1 Problem Statement

Academic and personal productivity work includes repetitive activities such as converting lecture PDFs into machine-readable text, reviewing email, checking deadlines, and combining tasks with calendar information. Existing automation platforms can solve parts of this problem, but they often require preconfigured workflows, broad platform adoption, or subscriptions. The project therefore explored whether a small natural-language agent could connect these tasks while retaining explicit safety controls.

1.2 Project Objectives

- Provide a Discord-based natural-language interface that is accessible only to the configured owner.
- Deliver an OCR-first MVP that creates LLM-readable text files from PDFs stored in OneDrive.
- Add practical Gmail, Google Sheets, and Google Calendar integrations.
- Demonstrate continuous operation on a MacBook Air M1 without Docker, a local LLM, MCP, browser automation, or a multi-agent framework.
- Keep test and demonstration API usage below KRW 10,000.

2. Scope and Requirements

OCR was prioritized as the required vertical slice. Gmail management and the daily briefing were implemented after the OCR path was proven. The scope intentionally excluded persistent agent memory, arbitrary shell execution, hardware control, local-model deployment, and multi-user support.

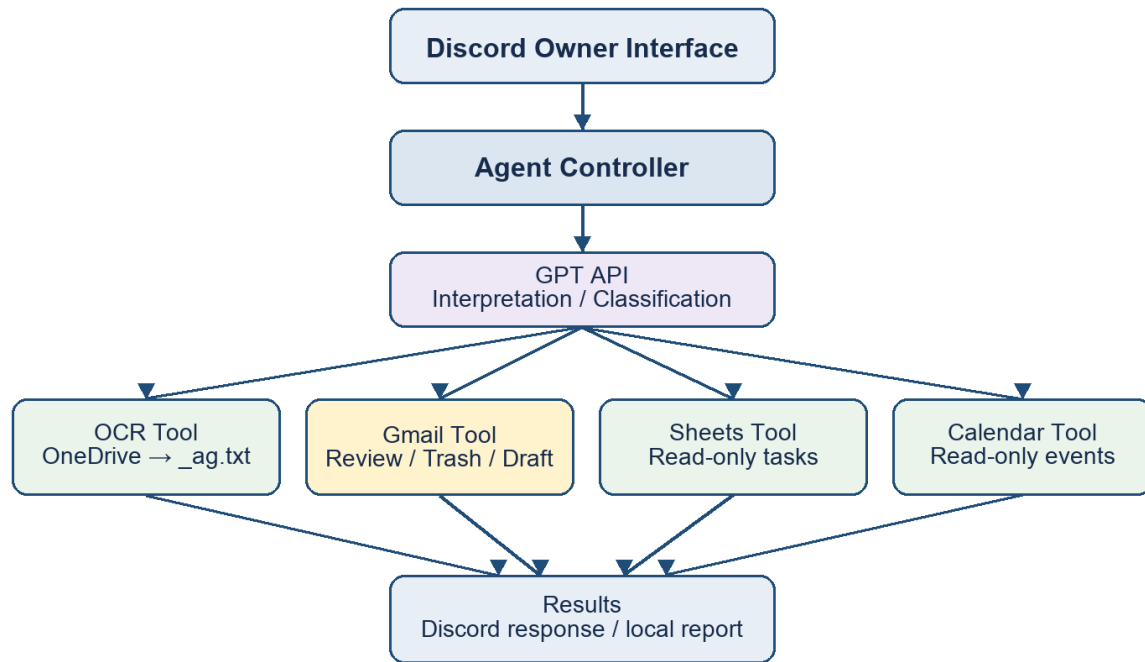
Area	Requirement	Result
Interface	Discord natural-language messages	Implemented
File source	OneDrive-mounted local directory only	Implemented
OCR output	UTF-8 TXT beside source, `_ag` suffix	Implemented
OCR content	Korean, English, math, tables, figures	Implemented
Gmail	Date-bounded review, summary, safe Trash	Implemented
Email send	Only after explicit user confirmation	Implemented
Calendar	Read only	Implemented
Task records	Google Sheets read only	Implemented
Memory	No database or long-term conversation memory	Implemented
Security	Owner restriction and prompt-injection defense	Implemented

2.1 Explicitly Excluded Functions

- OpenClaw repository installation or reuse
- MCP and multi-agent architecture
- Browser automation and arbitrary terminal commands
- Calendar create, update, or delete operations
- Discord file transfer
- Long-term conversational memory or database-backed state

3. System Architecture

The architecture separates probabilistic language-model work from deterministic privileged actions. The model interprets commands, transcribes page images, classifies email, and drafts summaries. Python code—not the model—controls paths, Google API calls, output creation, confirmation state, and allowlisted tool selection.



Model output is validated before deterministic tools execute.

Figure 1. Implemented system architecture and trust boundary.

3.1 Main Components

- **Discord Bot:** receives owner messages, rejects attachments, acknowledges processing, and splits long responses to fit Discord limits.
- **Agent Controller:** routes validated intents and maintains only short-lived OCR selection and Gmail confirmation state.
- **OCR Tool:** indexes PDF names inside the configured OneDrive root, renders selected pages, and sends images to the GPT vision-capable endpoint.
- **Gmail Tool:** performs date-bounded review, structured classification, conservative Trash actions, reply drafting, and explicit send/cancel confirmation.
- **Sheets and Calendar Tools:** read task records and upcoming events without write permissions.
- **launchd Service:** starts at login and restarts the bot if the process exits.

4. Functional Implementation

4.1 Discord Natural-Language Interface

The bot uses both deterministic phrase matching and structured GPT parsing. Common commands are routed without an additional model call, while less predictable wording is mapped to a fixed intent schema. Only the configured 19-digit Discord user ID is accepted. Each accepted request receives an immediate “Command received. Processing...” acknowledgement.

4.2 PDF OCR Automation

1. Search recursively within the configured OneDrive root using normalized fuzzy filename matching.
2. Ask the owner to select a candidate when the file reference is ambiguous.

3. Validate the requested page range and enforce the maximum pages per run.
4. Render each PDF page to a JPEG image and transcribe it with the GPT API.
5. Write Korean and English text, LaTeX mathematics, Markdown tables, figure descriptions, and page separators.
6. Create ``<original_name>_ag.txt`` atomically beside the source PDF.

Overwrite guarantee

The tool checks for an existing output before OCR and opens the final output in exclusive-create mode. If a matching ``_ag.txt`` already exists—or appears while processing—the operation stops without replacing it.

4.3 Gmail Management

Gmail organization requires an explicit range such as “last 48 hours” or “2026-06-11 to 2026-06-12.” If the owner omits the range, the bot asks for it and performs no mailbox action. Email is classified as important, schedule-related, promotional, or other. Only unmistakable promotional messages may be moved to Gmail Trash.

- Protected categories include personal, work, school, account, payment, receipt, authentication, security, legal, deadline-related, and ambiguous email.
- Injection-like text inside an email prevents automatic Trash movement.
- The Discord response reports the reviewed count, Trash count, selected subjects, and important summaries.
- A Markdown report named ``important_emails.md`` is saved locally for evidence and review.

4.4 Reply Draft and Approval

The agent can search for an email within an optional date range and generate a concise English reply draft. The draft is not sent immediately. It is assigned a short ID and displayed in Discord. The owner must enter ``send <ID>`` to transmit it or ``cancel <ID>`` to discard it. Pending drafts are held only in memory and disappear after restart.

4.5 Daily Productivity Briefing

The daily briefing combines important or schedule-related Gmail messages from the last 48 hours, unfinished Google Sheets tasks, and primary Google Calendar events from the next 48 hours. Completed rows and formula-only artifacts such as D+9699 are excluded. The final briefing is delivered directly in Discord rather than saved or attached as a file.

5. Security and Safety Design

Security principle

The LLM may interpret data, but it cannot directly execute shell commands, select arbitrary tools, escape configured paths, or send an email without the deterministic controller’s permission.

- Owner-only Discord authorization using a configured user ID.
- Filesystem access restricted to one resolved OneDrive root; path traversal and symlink escape are rejected.
- PDF-only OCR input and Discord attachment rejection.

- Fixed Pydantic intent and decision schemas rather than free-form tool instructions.
- Untrusted PDF, email, Sheet, and Calendar content wrapped with explicit data-only delimiters.
- No use of `eval`, `exec`, model-generated shell commands, or arbitrary browser control.
- OpenAI API requests use `store=False`.
- Secrets remain in `.env`, `credentials.json`, and `token.json`, which are excluded from submission and restricted to owner-only filesystem permissions.

6. Deployment and Continuous Operation

The application runs in a Python virtual environment on macOS. Google services use OAuth desktop-app credentials. A launchd LaunchAgent includes `RunAtLoad` and `KeepAlive`, allowing the Discord bot to start automatically at login and restart after failure. AC-power system sleep was disabled so the Mac can remain available continuously while connected to power and the internet.

- Hardware: MacBook Air M1, 8 GB RAM, 256 GB SSD
- Runtime: Python, discord.py, OpenAI Python SDK, PyMuPDF, Google APIs
- Storage: OneDrive-mounted local filesystem plus local logs/reports
- Operational status: AgenticAI_Bot connected to Discord and managed by launchd

7. Testing and Results

Testing combined automated unit tests with live end-to-end verification. The automated suite covers owner restriction, path safety, atomic output creation, PDF indexing, unreadable OneDrive placeholders, command routing, Gmail protection rules, date parsing, report writes, task completion filtering, and controller behavior.

Test	Input / Scope	Observed result	Status
OCR: timetable PDF	1 page	TXT generated; UTF-8/page marker verified	Pass
OCR: engineering math HW#1	1 page	Korean and LaTeX formulas preserved	Pass
Overwrite prevention	Repeated output path	Existing `_ag.txt` blocked before write	Pass
Discord access control	Authorized account	Command received and processed	Pass
Gmail organization	2026-06-11 to 2026-06-12	10 reviewed; 7 promotional messages moved to Trash	Pass
Email report	Gmail result	`important_emails.md` generated	Pass
Reply approval	Date-bounded email search	Draft generated; cancel flow confirmed	Pass
Daily briefing	Gmail + Sheets + Calendar	Direct Discord response	Pass
Automated test suite	21 tests	All tests passed	Pass
Persistent operation	launchd service	Running with KeepAlive and RunAtLoad	Pass

7.1 OCR Output Quality

The engineering mathematics assignment test preserved Korean instructions and mathematical expressions such as complex numbers, roots, powers, and exponential notation in LaTeX form. The timetable test verified general structured text extraction. Both outputs used the required `\_ag.txt` suffix and contained explicit page markers.

7.2 Gmail Live Result

For the inclusive range June 11–12, 2026, the agent reviewed 10 inbox messages, moved 7 unmistakable promotional messages to Trash, retained important content, summarized a Coursera course message, and reported the results in Discord. A subsequent date-bounded reply-draft command successfully produced a draft and supported cancellation.

7.3 Observed Corrections During Integration

Live Discord testing exposed configuration and routing problems that unit-only testing would not have found, including a mismatched bot application, a server/channel setup error, and an undefined variable in the Gmail date-range path. Each problem was corrected, converted into a regression test where applicable, and redeployed through the persistent service.

8. Comparison with Existing Services

The comparison below uses the project’s March 2026 frame and focuses on architectural fit rather than declaring a universal winner. Commercial products offer much broader integration catalogs and organizational governance. OpenClaw provides a more general personal-agent runtime. This project intentionally trades breadth for inspectability, a small attack surface, and exact control over the demonstrated workflows.

System	Deployment	Primary strength	Project-relevant trade-off
OpenClaw	Self-hosted personal agent	Chat apps; broad tools; persistent context	High flexibility; broader privilege and setup surface
Zapier Agents	Managed SaaS	Specialized agents across 9,000+ apps	Fast integration breadth; platform-dependent
n8n AI	Cloud or self-hosted workflows	Visual orchestration and AI agent nodes	Strong audit/human checkpoints; more workflow configuration
Microsoft 365 Copilot	Microsoft-managed service	Work-context chat and agents across Microsoft 365	Strong governance; centered on Microsoft ecosystem
This project	Local Python service on macOS	Discord + OCR + Gmail + Sheets + Calendar	Narrow scope with explicit permissions and deterministic guardrails

Interpretation: the implemented MVP is not a replacement for these platforms. Its contribution is a transparent, narrow implementation that demonstrates the same core agentic loop—understand, select a tool, act, and report—while making high-risk decisions explicit.

9. Limitations and Future Work

- The system depends on cloud APIs and internet connectivity.
- OneDrive online-only placeholders must be downloaded before OCR.
- OCR is processed page by page, so long documents increase cost and latency.

- Email classification remains probabilistic despite conservative deterministic checks.
- Pending confirmations are stored only in memory and are lost after process restart.
- The Mac must remain powered, logged in, connected, and awake.
- Python 3.9 currently produces dependency support warnings; migration to Python 3.11+ is recommended.

9.1 Recommended Extensions

- Persistent, encrypted approval state with expiration times
- Per-action audit log and cost/latency dashboard
- Batch OCR with resumable checkpoints and quality scoring
- Calendar creation behind explicit confirmation
- Automated health checks and failure notifications
- Evaluation dataset for OCR accuracy and email false-positive rates

10. Conclusion

The project achieved its objective of demonstrating a practical Agentic AI system on consumer hardware. A Discord message can now trigger real work across local files and Google services, while deterministic code controls permissions and sensitive actions. The OCR-first strategy produced an early demonstrable success, and the same architecture was extended to Gmail organization, approval-based replies, task records, calendar inspection, and daily briefings.

The final system shows that a useful personal automation agent does not require a large framework or local model deployment. A small controller, a hosted reasoning model, narrowly scoped tools, explicit user approval, and careful filesystem and content boundaries were sufficient to produce a functioning MVP. The most important outcome is therefore both functional and architectural: meaningful automation can be achieved quickly without surrendering all control to an unrestricted autonomous runtime.

References

- [1] **OpenClaw — Personal AI Assistant.** Official product page describing chat-based inbox, email, calendar, and personal-assistant automation. <https://openclaw.ai/>
- [2] **Zapier Agents.** Official page describing specialized agents and integrations across more than 9,000 applications. <https://zapier.com/agents>
- [3] **n8n AI Workflow Automation.** Official page emphasizing business rules, auditability, and human-in-the-loop checkpoints. <https://n8n.io/ai/>
- [4] **Microsoft 365 Copilot.** Official page describing work-context chat, agents, Microsoft 365 integration, and governed access. <https://www.microsoft.com/en-us/microsoft-365-copilot>
- [5] **Project source code and implementation specification.** Primary evidence for implemented behavior, tests, deployment, and live results. [Local project workspace, June 20–21, 2026](#)

Appendix A. Demonstration Commands

```
help
OCR pages 1-3 of modern control systems
Organize my emails
last 48 hours
```

Organize my emails from 2026-06-11 to 2026-06-12

Draft a reply to the email from Coursera from 2026-06-11 to 2026-06-12

cancel <ID>

Show my calendar

Create my daily briefing

Appendix B. Submission Security Checklist

- Include source code, README, implementation specification, requirements, `.env.example`, proposal, final report, and demonstration video.
- Exclude `.env`, `credentials.json`, `token.json`, `.venv`, runtime logs, and personal OCR source files.
- Reset any credential that has been exposed outside the local machine.
- Verify the Discord bot is either intentionally left running or stopped after the presentation.